

Architecture Frontend PNPI

Plateforme Nationale de Pilotage Industriel — Ministère de l'Industrie du Gabon Document de référence pour l'équipe technique et les nouveaux contributeurs. Version : lot 79 (avril 2026).

1. Vue d'ensemble

La PNPI utilise **Next.js 14.2** en mode **App Router** avec **React 18** et **TypeScript strict**. L'objectif d'architecture est triple :

1. Servir des pages **rapides au premier rendu** (Server Components par défaut, data fetch en parallèle cote serveur).
2. **Isoler le client** du backend FastAPI : aucun composant ne doit attaquer directement `http://localhost:8000` . Tout passe par un proxy Next.js.
3. Conserver une **identité visuelle ministérielle** cohérente : palette drapeau Gabon, typographie Playfair Display + Inter, mode sombre optionnel.

```
frontend/  
  src/  
    app/                # App Router (pages + layouts + components)  
      layout.tsx        # Shell unique (nav, theme, breadcrumbs)  
      page.tsx          # Vitrine publique  
      api/[...path]/    # Proxy catch-all vers FastAPI  
      pnpi/             # Espace agents (ministre, directeur, instructeur)  
      inspecteur/       # Espace inspecteur  
      profil/           # Profil utilisateur  
      admin/            # Administration (admin uniquement)  
      open-data/        # Open Data (public)  
      aide/guides/[role] # Guides PDF par role  
      components/       # Composants partagés (KPI, MegaNav, etc.)  
      hooks/            # Hooks custom (useDebounce, useForm, ...)  
      lib/              # Helpers (api.ts, backend.ts, role-routing.ts)  
      public/           # Logos, manifest, favicons  
      tests/e2e/        # Suites Playwright
```

2. Strategie de fetching

2.1 Server Components par default

Toutes les pages sont des **Server Components asynchrones** sauf cas justifie (state local, hooks, listeners). Pattern type :

```
export default async function Page() {
  let profile;
  try {
    profile = await fetchBackendProfile();
  } catch {
    redirect("/connexion");
  }

  const [a, b] = await Promise.all([
    fetchPNPIKpis().catch(() => null),
    fetchPNPIATIs({ limit: 20 }).catch(() => []),
  ]);
  // ...
}
```

Regles : - Les fetchs paralleles utilisent `Promise.all` avec `.catch(() => fallback)` pour eviter qu'une seule erreur casse toute la page. - L'auth (`fetchBackendProfile`) reste **avant** les fetchs metier, pour renvoyer vers `/connexion` au plus tot. - Le pattern defensif `Array.isArray(data)` ? `data : []` est obligatoire sur toute reponse de liste (cf. `CLAUDE.md`).

2.2 Proxy catch-all `/api/[...path]`

`frontend/src/app/api/[...path]/route.ts` forward toute requete `/api/*` vers le backend FastAPI en injectant le `Authorization: Bearer <token>` depuis le cookie `httpOnly npni_access_token` . Avantages :

- Zero token cote client : impossible de fuiter via XSS.
- Aucun mix HTTP/HTTPS en prod : tout passe par le meme domaine.
- Permet aux Server Components ET aux composants client d'utiliser la meme URL.

Exceptions : `/api/auth/*` et `/api/admin/impersonate/*` ont leurs propres routes pour gerer la pose/suppression de cookies.

2.3 Helpers `lib/`

- `lib/backend.ts` : `backendRequest(path, init?)` (Server Components, ajoute le cookie automatiquement) et `fetchBackendProfile()` .

- `lib/api.ts` : wrappers metier typés (`fetchPNPIKpis` , `fetchPNPIATIs` , `fetchPNPIInspections` , etc.).
- `lib/role-routing.ts` : `getDefaultRouteForRoles(roles)` et `getMegaNavForRoles(roles)` . Centralise la mappping role -> route + nav.

3. Composants reutilisables

Trente composants partagés sous `frontend/src/app/components/` (cf liste exhaustive dans `CLAUDE.md`). Les plus structurants :

COMPOSANT	ROLE
<code>MegaNav</code>	Mega-menu desktop par role, panels avec sections
<code>MobileNav</code>	Drawer mobile (hamburger) + overlay
<code>RepubliqueBand</code>	Bandeau "Republique gabonaise" en haut
<code>Breadcrumbs</code>	Generation auto depuis le pathname
<code>KpiCard</code>	Carte indicateur (variantes primary/success/neutral/accent)
<code>DataTable</code>	Table generique avec tri, pagination, scroll
<code>EmptyState</code>	9 illustrations SVG cohérentes (charte Gabon)
<code>Timeline</code>	Frise chronologique pour historique ATI
<code>StatusBadge</code>	Pastilles statut ATI (soumis, instruction, ...)
<code>Skeleton</code> / <code>SkeletonLoader</code>	Squelettes de chargement
<code>ConfirmDialog</code>	Modale de confirmation (RBAC sensitive)
<code>SignaturePad</code>	Capture signature inspecteur
<code>BriefingAudio</code>	Lecture vocale (TTS Web Speech API)
<code>ImpersonateBanner</code>	Banniere admin en mode "se connecter en tant que"
<code>ThemeToggle</code>	Switch clair/sombre (<code>data-theme</code> sur <code><html></code>)

4. Hooks custom

Sous `frontend/src/hooks/` :

- `useDebounce(value, delay)` — anti-rebond pour les filtres tape texte.
- `useForm({ initialValues, onSubmit, validate })` — formulaire controle avec validation synchrone, etat `submitting` , `errors` .
- `useLocalStorage(key, initial)` — persistance navigateur (preferences nav, filtres recents).
- `useMediaQuery(query)` — detection breakpoint cote client.
- `useOnlineStatus()` — listener `online` / `offline` (banniere PWA).
- `useIntersectionObserver(ref, opts)` — lazy-render des sections lourdes.
- `useKeyboardShortcut(combo, handler)` — raccourcis (CommandPalette).
- `useCopyToClipboard()` — copie + feedback toast.
- `usePagination(items, perPage)` — pagination cote client pour tables courtes.

5. Routage par role

L'utilisateur connecte est redirige vers son espace par default via `getDefaultRouteForRoles` :

ROLE	ROUTE PAR DEFAULT
<code>ministre</code> , <code>directeur</code> , <code>instructeur</code> , <code>admin</code>	<code>/pnpi</code>
<code>inspecteur</code>	<code>/inspecteur</code>
<code>operateur</code>	<code>/pnpi/guichet</code>
(non connecte)	<code>/connexion</code>

Le mega-menu est filtre par role : un operateur ne voit pas les actions d'instruction, un inspecteur ne voit pas le simulateur ministeriel, etc. La logique reste **cote serveur** (`layout.tsx`), donc aucun lien sensible n'apparait dans le HTML servi a un role non autorise.

Securite : chaque page Server Component **doit** appeler `fetchBackendProfile()` et verifier le role en tete. Le backend reverifie de toute facon (RBAC defense-in-depth).

6. Conventions de code

1. **Server Component par défaut.** `"use client"` uniquement si : - `useState`, `useEffect`, hooks de form, animations Framer Motion, listeners. - Composant ayant besoin du DOM (canvas, leaflet, recharts client).
2. **Pattern défensif `Array.isArray`** sur toute réponse de liste.
3. **Import absolu via `@/`** quand possible (`@/lib/api` , `@/components/KpiCard`).
4. **Pas d'URL backend en dur** cote client. Toujours `/api/...` ou `backendRequest(...)` cote serveur.
5. **Pas de `any` sauf cas justifié.** Les schemas Pydantic sont mappés en types TypeScript dans `lib/api.ts` .
6. **CSS via `globals.css` + variables.** Pas de Tailwind (choix institutionnel). Variables `--gabon-green` , `--pnpi-blue` , `--ink` , `--surface` , `--line` , etc. Ne jamais hardcoder `#fff` ou `#000` directement (utiliser `var(--bg-layer)` et `var(--ink)`).
7. **Pas d'emojis dans le code source** (charte ministérielle).

7. Performance

- **Lazy-load Leaflet** via `next/dynamic` avec `ssr: false` (il dépend du DOM). Idem pour `BriefingAudio` qui charge l'API Speech.
- **Skeleton fallback** via `loading.tsx` cote App Router : chaque page lourde qui fait des `await fetch()` a son skeleton (`SkeletonDashboard` , `SkeletonKpiCard` , `SkeletonLoader`).
- **`fetch` Server Side avec cache:** la valeur `cache: "no-store"` est utilisée pour les pages dynamiques (KPI temps réel) ; les pages statiques (open-data, aide) peuvent bénéficier de `revalidate: N` .
- **Bundle splitting :** Recharts et Leaflet sont importés uniquement là où ils servent (`next/dynamic`).
- **Image optimization :** `<Image>` de Next pour les logos et illustrations PNG ; les SVG sont inlineés ou référencés en `<img>` simple.

8. Mode sombre

Activation : `<html data-theme="light|dark">` , modifiée par `ThemeToggle.tsx` . Persiste en `localStorage` (`pnpi-theme`).

Strategie CSS : - Les variables (`--bg-base` , `--surface` , `--text-main` , `--line`) sont redefinies sous `html[data-theme="dark"]` dans `globals.css` . - Les composants utilisant les variables basculent automatiquement. - Les inline-styles avec couleurs hardcodees (`#fff`, `#f3f4f6`) sont contre-balances par des regles `[data-theme="dark"] [style*="..."]` dans le bloc "LOT 79 — DARK MODE COMPLEMENT" de `globals.css` .

A faire pour ajouter une page au mode sombre : 1. Privilegier `var(--surface)` , `var(--text-main)` , `var(--line)` . 2. Si l'inline-style est inevitable, ajouter une regle `html[data-theme="dark"] .ma-classe { ... }` dans `globals.css` .

Voir `docs/architecture/frontend-darkmode.md` pour le diagnostic page-par-page.

9. Accessibilite (cible WCAG 2.2 AA)

- Skip-link `#main-content` en haut du body (visible au focus clavier).
- Focus visible : `:focus-visible` avec outline ambre PNPI sur tous les composants interactifs.
- ARIA : `aria-label` sur les boutons icon-only, `aria-hidden="true"` sur les decoratifs, `role="banner|main|contentinfo"` sur la structure.
- Contraste : verifie sur la palette principale (texte sur fond), mais a re-auditer apres tout ajout d'inline-style.
- Reduced-motion : `@media (prefers-reduced-motion: reduce)` desactive les animations (`reveal-up` , `shimmer`).
- Composant `AccessibilityPanel` : permet a l'utilisateur d'augmenter la taille de texte, activer un contraste eleve, desactiver les animations.

10. Tests E2E

Suite Playwright sous `frontend/tests/e2e/` : - `auth.spec.ts` — login + redirection role. - `ati-flow.spec.ts` — soumission ATI operateur -> instruction -> approbation. - `inspections.spec.ts` — creation inspection + signature + cloture. - `accessibility.spec.ts` — `axe-core` sur les pages cles. - `darkmode.spec.ts` — switch theme + capture visuelle.

Commandes : `npm run test:e2e` (headless), `npm run test:e2e:ui` (interactif).

Annexes

- **Charte graphique :** `docs/charte-graphique.html`
- **Glossaire ATI :** `docs/glossaire-ati.html`
- **Securite RBAC :** `docs/rbac_matrix.md`
- **Deploiement :** `docs/deployment-guide.md`
- **Patterns CLAUDE :** `CLAUDE.md` racine du repo

Document maintenu par l'equipe technique. Toute modification d'un pattern structurel (proxy, role-routing, layout shell) doit etre relfetee ici.